MATH 189 - Data Analysis

Dani Nguyen

Winter 2025

Contents

1 Statistical Learning

1.1 Terminology

"Supervised Learning" means we have both predictors and responses.
"Predictors/input variables/features" are arranged in a vector.

$$X = (X_1, X_2, ..., X_n)$$

"Output variables/responses" is a scalar where $y \in \mathbb{R}$
We think of data as coming from a model.

$$y = f(x) + \epsilon$$

$\begin{cases} f(x) \text{ is fixed but unknown function} \\ \epsilon \text{ is the random error term with mean zero, independent of X} \end{cases}$

Question: Why estimate f?
Prediction: Given X, we want to predict y with $\hat{y} = \hat{f}(x)$

Mean square error is

$$\begin{aligned} \mathbb{E}(y - \hat{y})^2 &= \mathbb{E}(f(x) + \epsilon - \hat{f}(x)) \\ &= \mathbb{E}(f(x) - \hat{f}(x))^2 + 2\mathbb{E}((f(x) - \hat{f}(x))\epsilon) + \mathbb{E}(\epsilon^2) \end{aligned}$$

Examine the middle term in detail

$$2\mathbb{E}((f(x) - \hat{f}(x))\epsilon) = \mathbb{E}(f(x) - \hat{f}(x)) \cdot \mathbb{E}(\epsilon) = 0$$

We know that

$$\mathbb{E}(y - \hat{y})^2 = (f(x) - \hat{f}(x))^2 + \mathbb{E}(\epsilon^2)$$

Where the left term is reducible error and the right term is irreducible error.

"Inference" - Sometimes we want to understand the association between y and $X_1, ..., X_p$

1. Which predictors are associated with response?
2. Is the association positive or negative?
3. Is the relationship approximately linear?

1.2 Accuracy

Training data

$$(x_1, y_1), (x_2, y_2), ...(x_n, y_n)$$
$$\text{where } X_i = (X_{i1}, X_{i2}, ..., X_{ip})$$

n = # data points (e.g. $n = 30$)
p = # predictors (e.g. $p = 3$)

Mean Square Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2$$

Defined on training data

$$\text{Test MSE} = \text{ Ave } (y_0 - \hat{f}(x_0))^2$$

Defined on previously unseen data point (x_0, y_0)

Linear / affine model

$$f(x) = \beta_0 + \beta_1 x_1 + ... + \beta_p x_p$$

"Fit/train the model" usually means run an optimization on your computer that uses training data and outputs the parameters $\beta_0, \beta_1, ..., \beta_p$ that make the MSE and hopefully the test MSE small.

Classification
Data set $(x_1, y_1), (x_2, y_2,), ..., (x_n, y_n)$ where $X_i = (x_{i1}, ..., x_{ip})$ and y_i is qualitative.

Error rate is the fraction of incorrect classifications.

$$\frac{1}{n}\sum_{i=1}^{n} \mathbb{I}\{y_i \neq \hat{y}_i\}$$

$$\mathbb{I}\{y_i \neq \hat{y}_i\} = \begin{cases} 1, & y_i \neq \hat{y}_i \\ 0, & y_i = \hat{y}_i \end{cases}$$

Test error rate

$$\text{Ave}(\mathbb{I}\{y_0 \neq \hat{y}_0\})$$

Where (x_0, y_0) is a previously unseen test data point.

K-nearest neighbors (KNN)
The KNN classifier is defined by letting $\eta_0 =$ set of indices $i \in \{1, ..., n\}$ for K nearest neighbors of X_0 among the input data.

$\hat{y}_0 = j$ where j maximizes

$$\sum_{i \in \eta_0} \mathbb{I}\{y_i = j\}$$

which is the number of nearest neighbors (out of K) assigned label j.

2 Linear Regression

2.1 Simple Linear Regression

Underlying statistical model: $y = \beta_0 + \beta_1 X + \epsilon$ where y is the quantitative response and a single predictor X which is why it is called simple.

Prediction model: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 X$
$\hat{\beta}_0, \hat{\beta}_1$ are estimates based on training data, where hat means estimated quantity.

Estimating coefficients
The residual sum of squares (RSS)

$$\begin{aligned} \text{RSS} &= \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2 \end{aligned}$$

Choose $\hat{\beta}_0, \hat{\beta}_1$ to minimize RSS.

$$\begin{aligned} \frac{\partial \text{RSS}}{\partial \hat{\beta}_0} &= \frac{\partial}{\partial \hat{\beta}_0} \sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 X)^2 \\ &= 2 \sum_{i=1}^{n} (\hat{\beta}_0 + \hat{\beta}_1 X_i - Y_i) \\ &\implies \text{RSS is } \begin{cases} \text{decreasing for really small } \hat{\beta}_0 \\ \text{increasing for really big } \hat{\beta}_0 \end{cases} \end{aligned}$$

RSS is optimal (in $\hat{\beta}_0$) when

$$\begin{aligned} 0 &= \frac{1}{n} \sum_{i=1}^{n} (\hat{\beta}_0 + \hat{\beta}_1 X_i - Y_i) \\ &= \hat{\beta}_0 + \hat{\beta}_1 \Big(\frac{1}{n} \sum_{i=1}^{n} X_i \Big) - \Big(\frac{1}{n} \sum_{i=1}^{n} Y_i \Big) \\ &= \hat{\beta}_0 + \hat{\beta} \bar{x} - \bar{y} \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x} \end{aligned}$$

Where $\begin{cases} \bar{x} = \frac{1}{n} \sum_{i=1}^{n} X_i \text{ is the mean predictor} \\ \bar{y} = \frac{1}{n} \sum_{i=1}^{n} Y_i \text{ is the mean response} \end{cases}$

Covariance

$$\begin{aligned}\mathrm{Cov}(X_i, Y_i)_{i=1}^n &= \frac{1}{n}\sum_{i=1}^n X_i Y_i - \bar{X}\bar{Y} \\ &= \frac{1}{n}\sum_{i=1}^n (X_i - \bar{x})(Y_i - \bar{y})\end{aligned}$$

Variance

$$\begin{aligned}\mathrm{Var}(X_i)_{i=1}^n &= \mathrm{Cov}(X_i, X_i)_{i=1}^n \\ &= \frac{1}{n}\sum_{i=1}^n (X_i - \bar{x})^2\end{aligned}$$

From this we can get our optimal $\hat{\beta}_1$

$$\begin{aligned}\hat{\beta}_1 &= \frac{\mathrm{Cov}(X_i, Y_i)_{i=1}^n}{\mathrm{Var}(X_i)_{i=1}^n} \\ &= \frac{\frac{1}{n}\sum_{i=1}^n (X_i - \bar{x})(Y_i - \bar{y})}{\frac{1}{n}\sum_{i=1}^n (X_i - \bar{x})^2}\end{aligned}$$

2.2 Goodness of Fit & Correlation

Assume an underlying statistical model

$$y = \beta_0 + \beta_1 X + \epsilon$$

We use training data $(X_i, Y_i)_{i=1}^n$ to generate prediction model $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 X$
The formulas for $\hat{\beta}_0$ and $\hat{\beta}_1$ to minimize RSS:

$$\mathrm{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \begin{cases} \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{X} \\ \hat{\beta}_1 = \frac{\mathrm{Cov}(X_i, Y_i)_{i=1}^n}{\mathrm{Var}(X_i)_{i=1}^n} \end{cases}$$

Goodness of fit

$$\begin{aligned}
\text{RSS} &= \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \\
&= \sum_{i=1}^{n}(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2 \\
&= \sum_{i=1}^{n}\Big((y_i - \bar{y}) - \hat{\beta}_1(x_i - \bar{x})\Big)^2 \\
&= \sum_{i=1}^{n}(y_i - \bar{y})^2 - 2\hat{\beta}_1 \sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y}) + \hat{\beta}_1^2 \sum_{i=1}^{n}(x_i - \bar{x})^2 \\
&= \sum_{i=1}^{n}(y_i - \bar{y})^2 - \frac{\Big[\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})\Big]^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}
\end{aligned}$$

RSS scales with the number of data points n and also scales with y^2 units.
$\rightarrow$ We can fix the scaling and get out a goodness-of-fit score (R^2) between 0 and 1.

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$$

$$\text{RSS} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad\qquad \text{TSS} = \sum_{i=1}^{n}(y_i - \bar{y})^2$$

So for simple linear regression,

$$\begin{aligned}
R^2 &= 1 - \frac{\text{RSS}}{\text{TSS}} \\
&= \frac{\text{TSS} - \text{RSS}}{\text{TSS}} \\
&= \frac{\Big[\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})\Big]^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2} \\
R &= \frac{\text{Cov}((x_i), (y_i))}{\text{Var}(x_i)^{1/2}\text{Var}(y_i)^{1/2}}
\end{aligned}$$

Long story short R^2 is the fraction of variance explained by the model.

$\boxed{ex}$ $\hat{\text{sales}} = 7.0 + 0.048 \cdot \text{TV}$
$R^2 = 0.61 = 61\%$ $r \approx 0.78$ pretty strong positive correlation

2.3 Regression Table

$\boxed{ex}$

	Coefficient	Std. error	t-statistic	p-value
Intercept (β_0)	7.0	0.46	15	<0.0001
TV (β_1)	0.048	0.0027	18	<0.0001

Std error is the error bar on linear coefficients.

$$\text{We know that } \begin{cases} \beta_0 \approx \hat{\beta}_0 \pm \text{SE}(\hat{\beta}_0) \\ \beta_1 \approx \hat{\beta}_1 \pm \text{SE}(\hat{\beta}_1) \end{cases}$$

Assume the underlying statistical model is true!

$$y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$
where ϵ_i are independent $\eta(0, \sigma^2)$ random variables

If we generate N data points (x_i, y_i) from the model, then
$$\begin{cases} \text{Var}(\hat{\beta}_0)^{1/2} \approx \text{Std. } \hat{}\text{error}(\hat{\beta}_0) \\ \text{Var}(\hat{\beta}_1)^{1/2} \approx \text{Std. } \hat{}\text{error}(\hat{\beta}_1) \\ \mathbb{E}(\hat{\beta}_0) = \beta_0 \\ \mathbb{E}(\hat{\beta}_1) = \beta_1 \end{cases}$$

2.4 Multiple Linear Regression

The underlying statistical model that we assume is true.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_p x_p$$

where y is response, β_0 is the intercept, $\beta_1 x_1, ..., \beta_p x_p$ are predictors, and ϵ is the mean zero noise.

$\boxed{ex}$ $\text{sales} = \beta_0 + \beta_1 \text{TV} + \beta_2 \text{Radio} + \beta_3 \text{Newspaper} + \epsilon$

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + ... + \hat{\beta}_p x_p$$

We want to find the best $\hat{\beta}$ to minimize RSS, where $\hat{\beta}$ are estimated quantities.

$$\begin{aligned} RSS &= \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - ... - \hat{\beta} x_{ip})^2 \\ &= \left|\left| \begin{bmatrix} y_1 - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - ... - \hat{\beta}_p x_{ip} \\ \vdots \\ y_n - \hat{\beta}_0 - \hat{\beta}_1 x_{n1} - ... - \hat{\beta}_p x_{ip} \end{bmatrix} \right|\right|^2 \\ &= ||Y - X\hat{\beta}||^2 \end{aligned}$$

where $y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ $\hat{\beta} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \\ \vdots \\ \hat{\beta}_p \end{bmatrix}$ $X = \begin{bmatrix} 1 & x_{11} & \dots & x_{1p} \\ 1 & x_{21} & \dots & x_{1p} \\ & & \ddots & \\ 1 & x_{n1} & \dots & x_{np} \end{bmatrix}$

The best $\hat{\beta}$ to minimize Euclidean norm^2 for $||Y - X\hat{\beta}||^2$ is $\hat{\beta} = X^+ Y = (X^T X)^{-1} X^T Y$

	Coefficient	Std. error	t-statistic	p-value
Intercept	$\hat{\beta}_0$ 2.939	0.3119	9.42	<0.0001
TV	$\hat{\beta}_1$ 0.046	0.0014	32.81	<0.0001
Radio	$\hat{\beta}_2$ 0.189	0.0086	32.89	<0.0001
Newspaper	$\hat{\beta}_3$ -0.001	0.0059	-0.18	0.8599

Note: the standard error is NOT $Var(\hat{\beta}_i)^2$ but rather $\hat{Var}(\hat{\beta}_i)^2$

Hypothesis testing

State a hypothesis, test it with data (testing the likelihood such data could possibly be generated if the hypothesis were true).

H_0 - the null hypothesis: $y = \beta_0+\beta_1 x_1+\beta_2 x_2+\beta_3 x_3+\epsilon$ for some coefficients B_0, β_2, β_3 with $\beta_1 = 0$ with $\epsilon \sim N(0, \sigma^2)$ for some variance $\sigma^2 \geq 0$. Also, observations are independent.

H_1 - the alternative hypothesis: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \epsilon$ where everything above holds except $\beta_1 \neq 0$

Assume H_0. Then

$$\frac{\hat{\beta}_1}{\hat{SE}(\hat{\beta}_1)} = t \sim T_{n-p-1}$$

3 Logistic Regression & Generative Models

3.1 Logistic Regression

We want to predict whether an event happens ($y = 1$) or not ($y = 0$), we really want to model the probability $p \in (0, 1)$ of occurrence $\implies$ NOT LINEAR REGRESSION.
Before

$$y = \beta_0 + \beta_1 X_\epsilon$$

Now

$$\begin{aligned} p &= \frac{\exp(\beta_0 + \beta_1 X)}{1 + \exp(\beta_0 + \beta_1 X)} \\ 1 - p &= \frac{1}{1 + \exp(\beta_0 + \beta_1 X)} \\ \frac{p}{1-p} &= \exp(\beta_0 + \beta_1 X) \end{aligned}$$

Where the term $\frac{p}{1-p}$ is called "odds".
Note: $\log(\text{odds}) = \beta_0 + \beta_1 X$

$\boxed{ex}$ Defaulting on bank loans

$$\begin{aligned} p &= \text{probability of defaulting} \\ X &= \text{balance} \\ y_i &= \begin{cases} 0 & \text{default doesn't happen} \\ 1 & \text{default does happen} \end{cases} \\ \log(\frac{p}{1-p}) &= \beta_0 + \beta_1 \cdot \mathrm{X} \end{aligned}$$

How do we use data/observations $(x_i, y_i)_{1 \le i \le n}$ to bestimate β_0, β_1.

$$\begin{cases} \text{Linear regression} \implies \text{minimize RSS} \\ \text{Logistic regression} \implies \text{use max likelihood, do it on a computer} \\ \text{Generative models} \implies \text{simple formulas for } \hat{\Pi}_k, \hat{\mu}_k, \hat{\Sigma}_k \end{cases}$$

We choose estimates $\hat{\beta}_0, \hat{\beta}_1$ to maximize likelihood of observations $(x_i, y_i)_{1 \le i \le n}$ if (a) the observations were independent and (b) they were generated exactly from the statistical model.

$$\begin{aligned} \mathbb{P}\left((x_i, y_i)_{1 \le i \le n}\right) &= \prod_{i=1}^{n} \mathbb{P}(x_i, y_i) \text{ where } y_i \in \{0, 1\} \\ &= \prod_{y_i=1} \hat{p}(x_i) \prod_{y_i=0} (1 - \hat{p}(x_i)) \\ &= \prod_{i:y_i=1} \frac{\exp(\beta_0 + \beta_1 X_i)}{1 + \exp(\beta_0 + \beta_1 X_i)} \prod_{i:y_i=0} \frac{1}{1 + \exp(\beta_0 + \beta_1 X_i)} \end{aligned}$$

Know that $\hat{\beta}_0, \hat{\beta}_1$ are chosen to maximize this probability/likelihood above.

$\boxed{ex}$ $P_{\text{defaulting}}$

	Coefficient	Std. error	z-statistic	p-value
Intercept	-10.6513	0.3612	-29.5	<0.0001
Balance	.0055	0.0002	24.9	<0.0001

We want $\hat{\beta}_0 + \hat{\beta}_1 X = 0$

$$\begin{aligned} -10.6513 + 0.0055X &= 0 \\ X &= \frac{10.6513}{0.0055} \\ &= 1936.6 \end{aligned}$$

Assume

a. The model is true

b. Observations $(x_i, y_i)_{1 \leq i \leq n}$ are independent draws from the model

c. Model is true with $\beta_0 = 0$ (null hypothesis)

3.2 Generative Models for Classification

Predictors
X or $X_1, X_2, ..., X_n$
Response
$y \in \{0, 1\}$ or $y \in \{1, 2, ..., k\}$

Review
Simple logistic regression

$$P = \text{prob}\{y = 1\} = \frac{\exp(\beta_0 + \beta_1 X)}{1 + \exp(\beta_0 + \beta_1 X)}$$

Note: This doesn't make any assumptions about how predictors are distributed.

For each $k = 1, 2, ..., K$:
Our statistical model ('mixture of Gaussians') says $y = k$ with probability π_k ($\sum_{k=1}^{K} \pi_k = 1$)
Given $y = k$, $X \sim N(\mu_k, \Sigma_k)$ where Σ is the covariance matrix symbol. Here is the general vector-valued format:

$$X = (X_1, ..., X - p) \qquad \Sigma_k = \begin{bmatrix} & \end{bmatrix} \text{ pxp matrix}$$

Where μ_k has Kp parameters and Σ_k has $K\frac{p(p+1)}{2}$ parameters
When $p = 1$, we can write

$$X \sim N(\mu_k, \sigma_k^2)$$

In approaches like Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Naive Bayes (NB) use simple estimators for Π_k, μ_k, Σ_k that we can calculate fast.

$\boxed{ex}$ Linear discriminant analysis (LDA) make assumptions of common covariance structure $\Sigma = \Sigma_1 = \Sigma_2 = ... = \Sigma_k$ which reduces a lot of complexity. Given observations $(X_i, y_i)_{1 \le i \le n}$, we can estimate for $k = 1, ..., K$

$$\begin{aligned}
\hat{\Sigma} &= \frac{n_k}{n}, \text{ where} \\
n &= \#\{i : y_i = k\} \\
&= \text{number of observations in class } k \\
\hat{\mu}_k &= \frac{1}{n_k} \sum_{i:y_i=k} X_i
\end{aligned}$$

Last estimate common covariance by

$$\hat{\Sigma} = \frac{1}{n-k} \sum_{k=1}^{K} \sum_{i:y_i=k} (X_i - \mu_k)(X_i - \mu_k)^T$$

$$\mathbb{E}\hat{\Sigma} = \Sigma \text{ if the model is true}$$

$\boxed{ex}$ Quadratic discriminant analysis is pretty much the same except DIFFERENT covariances $\Sigma_1, ..., \Sigma_k$ for each class.
$\implies \hat{\Pi}_k = \frac{n_k}{n}$, $\hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} X_i$
$\implies \hat{\Sigma}_k = \frac{1}{n_k - 1} \sum_{i:y_i=k} (X_i - \mu_k)(X_i - \mu_k)^T$
$\implies K\frac{p(p+1)}{2}$ covariance free parameters.
$\boxed{ex}$ Naive Bayes is pretty much the same except DIFFERENT covariances $\Sigma_1, ..., \Sigma_k$ that are also diagonal matrices.
$\implies Kp$ covariance free parameters with the same estimators as QDA.

$$\text{diag}\left(\sum_k^n\right) = \frac{1}{n_k - 1} \sum_{i:y_i=k} (X_i - \mu_k) \odot (X_i - \mu_k)$$

Suppose you have the statistical model with probability Π_k, we set $y = k$ and draw $X \sim N(\mu_k, \Sigma_k)$. Suppose we have estimated the parameters well - see a new predictor vector $X = (X_1, ..., X_p)$ - How do you determine class probabilities $P\{y = k|X\}$ and make predictions?

Prediction part is easy - just predict based on the highest probability class: $\hat{y} = k$ for $k = \text{argmax}_{1 \leq j \leq k}\ P\{y = j|X\}$

So what is $P\{y = k|X\}$? Bayes' rule tells us

$$\begin{aligned} P\{y = k|X\} &= \frac{P\{x|y = k\}\ P\{y = k\}}{\sum_{j=1}^{k} P\{X|y = j\}P\{y = j\}} \\ &= P(X) \\ &= \frac{f_k(X)\Pi_k}{\sum_{j=1}^{K} f_j(X)\Pi_j} \end{aligned}$$

$$\text{where Gaussian density } \Pi_j(X) = \frac{1}{(2\pi)^{p/2}(\det \Sigma)^{1/2}} \exp(\frac{-1}{2}(X - \mu_j)\Sigma_j(X - \mu_j))$$

3.3 Discriminant Analysis In Detail

Let's say we have a generative statistical model with probability $\frac{1}{2}, y = 1$. Otherwise, $y = 2$.
Given $y = 1$, $X \sim N(\mu_1, \sigma^2)$
Given $y = 2$, $X \sim N(\mu_2, \sigma^2)$
We think about probability of class membership using Bayes' rule.

$$P\{Y = k|X\}$$

Where Y is response, k is class, either 1 or 2, and X is the predictor.

$$\begin{aligned} &\frac{P\{X, Y = k\}}{P\{X\}} \\ &= \frac{P\{X, y = k\}}{\sum_{j=1}^{2} P\{X, y = j\}} \\ &= \frac{P\{y = k\}P\{X|y = k\}}{\sum_{j=1}^{2} P\{y = j\}P\{X|y = j\}} \end{aligned}$$

Plugging in the specific model for class 1,

$$\begin{aligned} P\{y=1|X\} &= \frac{\frac{1}{2}f_1(X)}{\frac{1}{2}f_1(X)+\frac{1}{2}f_2(X)} \\ \text{where } f_1(x) &= P\{X|y=1\} \\ &= \frac{1}{\sqrt{2\pi\sigma^2}}\exp(-\frac{1}{2\sigma^2}(X-\mu_1)^2) \\ f_2(x) &= P\{X|y=2\} \\ &= \frac{1}{\sqrt{2\pi\sigma^2}}\exp(-\frac{1}{2\sigma^2}(X-\mu_2)^2) \end{aligned}$$

Usually our task in this class is to predict y as well as possible $\hat{y}=1$ or $\hat{y}=2$. The best we can do, the only normal strategy, is to predict the higher probability count.

$$\hat{y}=1 \text{ if } P\{y=1|X\} > P\{y=2|X\}$$

Which happens if and only if

$$\begin{aligned} \frac{1}{\sqrt{2\pi\sigma^2}}\exp(-\frac{1}{2\sigma^2}(X-\mu_1)^2) &> \frac{1}{\sqrt{2\pi\sigma^2}}\exp(-\frac{1}{2\sigma^2}(X-\mu_2)^2) \\ \exp(-\frac{1}{2\sigma^2}(X-\mu_1)^2) &> \exp(-\frac{1}{2\sigma^2}(X-\mu_2)^2) \\ -\frac{1}{2\sigma^2}(X-\mu_1)^2 &> -\frac{1}{2\sigma^2}(X-\mu_2)^2 \\ (X-\mu_2)^2 &> (X-\mu_1)^2 \end{aligned}$$

So we make the prediction, $\begin{cases} \hat{y}=1 & \text{if } |X-\mu_1| < |X-\mu_2| \\ \hat{y}=2 & \text{if } |X-\mu_2| < |X-\mu_1| \end{cases}$

We want to move from 1 predictor X to two predictors X_1 and X_2

<u>Before</u>
$P\{y=1\} = P\{y=2\} = \frac{1}{2}$
Given $y=1$, $X \sim N(\mu_1, \sigma^2)$
Given $y=2$, $X \sim N(\mu_2, \sigma^2)$

<u>After</u>

$P\{y=1\} = P\{y=2\} = \frac{1}{2}$
Given $y=1$, $X \sim N(\mu_1, \sigma^2)$
Given $y=2$, $X \sim N(\mu_2, \sigma^2)$
Where $\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$ and $\vec{\mu}_1 = \begin{bmatrix} \mu_{11} \\ \mu_{12} \end{bmatrix}$ and $\vec{\mu}_2 = \begin{bmatrix} \mu_{21} \\ \mu_{22} \end{bmatrix}$

Making it bold to turn scalars into vectors works except with variance.

$$\Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}$$

4 Model Selection

4.1 Bootstrap

Quantitative response - "train" different multiple linear regression models with different predictors.
Qualitative response - LDA / QDA / NB multiple logistic regression models with different predictors.

To get the best model, consider:

1. Set aside validation data set

$$\begin{array}{ccc} & (X_i, y_i)_{1 \le i \le n} & \\ & \swarrow \quad \searrow & \\ (X_i, y_i)_{i \in T} & & (X_i, y_i)_{i \in T^C} \end{array}$$

   Where T is a set of training data point indices, e.g. $T = \{1, ..., 100\}$ and T^C is the "test/validation" data points, e.g. $T^C = \{101, ..., 110\}$. $\hat{y}_i$ is the prediction for input X_i often using training data $(X_i, y_i)_{i \in T}$ to train the model. We can measure $\text{MSE}_{\text{test}} =$
   $\Rightarrow$ Choose the model with the best (lowest) MSE_{test}

2. Leave-one-out cross-validation (LOOCV)
   Idea: use all possible train-test splits.

$$(X_j, y_j)_{j \neq i}$$

3. K-fold cross validation
   Split your data according to k index sets $\{1, ..., n\} = S_1 \cup S_2 \cup ... \cup S_k$

4. Bootstrap

   $\boxed{ex}$ you invest $\begin{cases} \alpha & \text{fraction of your money in asset with return value } X \\ 1-\alpha & \text{fraction in asset with return value } y \end{cases}$

   You want to minimize variance of your return.
   Choose α to minimize

$$\text{Var}\left[\alpha X + (1-\alpha)y\right] = \text{Cov}[\alpha X + (1-\alpha)y, \alpha X + (1-\alpha)y]$$

   To optimize, set α derivative equal to zero.

$$\begin{aligned} \alpha &= \frac{\text{Cov[y-x, y]}}{\text{Var[y-x]}} \\ &= \frac{\sigma_y^2 - \sigma_{xy}}{\sigma_x^2 + \sigma_y^2 - 2\sigma_{xy}} \end{aligned}$$

   In practice, we use data $(x_i, y_i)_{1 \le i \le n}$ to estimate

$$\hat{\alpha} = \frac{\hat{\sigma}_y^2 - \hat{\sigma}_{xy}}{\hat{\sigma}_x^2 + \hat{\sigma}_y^2 - 2\hat{\sigma}_{xy}}$$

$$\text{where} \begin{cases} \hat{\sigma}_x^2 = \frac{1}{n-1}\sum_{i=1}^n (x_i - \bar{x})^2 \\ \hat{\sigma}_y^2 = \frac{1}{n-1}\sum_{i=1}^n (y_i - \bar{y})^2 \\ \hat{\sigma}_{xy} = \frac{1}{n-1}\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \end{cases}$$

   Q: What is a good error bar for $\hat{\alpha}$?
   Q: what is $\text{SE}[\hat{\alpha}] = \text{Var}[\hat{\alpha}]^{\frac{1}{2}}$?
   Bootstrap answers this.

   Bootstrap with $B(\approx 10^2 - 10^9)$ samples
   For $j = 1, ..., B$:
   Make fake data set $(\tilde{x}_i, \tilde{y}_i)_{1 \le i \le n}$ that comes from real observations $(x_i, y_i)_{1 \le i \le n}$ by uniformly sub-sampling with replacement.

4.2 Linear Model Selection

Goal: Choose best linear regression model based on predictor selection and modification of predictors.

Question: Given a data set with p predictors and a quantitative response variable, which subset of predictors leads to the best multiple linear regression model?

1. Best subset selection
    - Try all 2^p models and evaluate the MSE on a validation data set.
    - Choose the best one.
2. Forward selection
    - Start with an empty set of variables.
    - Try adding each of the remaining variables and calculate the MSE on a validation set.
    - Choose the best variable and add it to the model if it improves the MSE.
    - Repeat until convergence.
3. Backward selection
    - Start with all p variables.
    - Try kicking out each of the variables and calculate MSE on a validation set.
    - If it improves MSE, make the best alteration to the model.
    - Repeat until convergence.

Computational cost. The main advantage of forward and backward selection is computational cost. They require at most $p+(p-1)+...+1 = p(p+1)/2$ models, and

$$p(p+1)/2 \ll 2^p$$

Accuracy. There are no guarantees that forward or backward selection find the best model, but sometimes we get lucky. The models discovered by different methods can be slightly different.

Ridge regression. One way to improve the coefficients $\hat{\beta}_0, ..., \hat{\beta}_p$ that arise in a linear model is by changing the fitting procedure. Instead of minimizing the residual sum of squares

$$\text{RSS} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

over the data set $(x_i, y_i)_{1 \leq i \leq n}$, we choose $\hat{\beta}_0, ..., \hat{\beta}_p$ to minimize

$$\text{RSS} + \underbrace{\lambda \sum_{j=1}^{p} \hat{\beta}_j^2}_{\text{shrinkage penalty}}$$

The shrinkage penalty keeps the coefficients $\hat{\beta}_1, ..., \hat{\beta}_p$ small in magnitude, note that we don't penalize the intercept $\hat{\beta}_0$. A couple limits are worth considering.

- When $\lambda = 0$, we get multiple linear regression.
- When $\lambda \rightarrow$, all coefficients except the intercept approach zero.

It is best to apply ridge regression (or lasso) after standardizing each predictor by subtracting the mean and dividing by the standard deviation.

$$x_{ij} \rightarrow \tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)^2}} \text{ where } \bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij} \text{ is mean of predictor } j.$$

Lasso. Instead of minimizing $\text{RSS} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$, the lasso chooses $\hat{\beta}_1, ..., \hat{\beta}_p$ to minimize

$$\text{RSS} + \underbrace{\lambda \sum_{j=1}^{p} |\hat{\beta}_j|}_{\text{lasso penalty}}$$

Note that the lasso penalty uses the $\updownarrow_1$ norm $||\hat{\beta}||_1 = \sum_{j=1}^{p} |\hat{\beta}_j|$, whereas ridge regression uses the square $\updownarrow_2$ norm $||\hat{\beta}||_2^2 = \sum_{j=1}^{p} \beta_j^2$.

Principal component regression. *Principal component analysis* finds the directions in a data set $x_1, ..., x_n \in \mathbb{R}^p$ that have the most variance. Equivalently, principal component analysis finds a low-dimensional subspace that reconstructs the data as well as possible. After performing principal component analysis, we can try *principal component regression* (PCR), where we use a small number of principal components (e.g. 1-20) as predictors.

5 Nonlinearity

5.1 Polynomial Regression

Idea #1 Replace simple linear regression.

$$y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

with polynomial regression

$$y_i = \beta_0 + \beta_1 X_i + \beta_2 X_i^2 + ... + \beta_\alpha X_i^d + \epsilon_i$$

ex Wage vs age
We can separate out "high earners" and run logistic polynomial regression.

$$\mathbb{P}\{y_i > 250 | X_i\} = \frac{\exp(\beta_0 + \beta_1 X_i + ... + \beta_\alpha X_i^d)}{1 + \exp(\beta_0 + \beta_1 X_i + ... + \beta_\alpha X_i^d)}$$

Idea #2 Split a quantitative variable into an ordered categorical variable.
We have new predictors:

$$\begin{aligned} C_0(X) &= \mathbb{I}\{X < C_1\} \\ C_1(X) &= \mathbb{I}\{c_1 \leq X < c_2\} \\ &\vdots \\ C_{k-1}(X) &= \mathbb{I}\{C_{k-1} \leq X < C_k\} \\ C_k(X) &= \mathbb{I}\{C_k \leq X\} \end{aligned}$$

where $C_1, C_2, ..., C_k$ are cutpoints. Our book uses cutpoints $C_1 = 35, C_2 = 50, C_3 = 85$.

We can then run linear regression with

$$y_i = \beta_0 + \beta_1 C_1(X_i) + \beta_2 C_2(X_i) + ... + \beta_k C_k(X_i) + \epsilon_i$$

in this model, there are k degrees of freedom in this model.

Idea #3 Use piecewise polynomials (typically $d = 3$)

$$y_i - \begin{cases} \beta_{01} + \beta_{11}X_i + \beta_{21}X_i^2 + \beta_{31}X_i^3 + \epsilon_i & X_i < C \\ \beta_{02} + \beta_{12}X_i + \beta_{22}X_i^2 + \beta_{32}X_i^3 + \epsilon_i & X_i \geq C \end{cases}$$

This usually results in discontinuities within the data.

Idea #4 Use cubic splines
A cubic spline is a piecewise cubic model that imposes the continuity of

1. Prediction function
2. Its first derivative
3. Its second derivative

To find the number of free coefficients with k cutpoints / "knots" $\xi_1, \xi_2, ..., \xi_k$, we introduce a new predictor

$$\begin{aligned} h(X_i\xi_i) &= (X - \xi_i)^3 \\ &= \begin{cases} (X - \xi_i)^3 & X > \xi_i \\ 0 & x \leq \xi_i \end{cases} \end{aligned}$$

To fit cubic splines with k knots, we run multiple linear regression with an intercept and predictors $X, X^2, X^3, h(X_i\xi_1), ..., h(X_i\xi_k)$ with $k + 4$ free variables βs.

5.2 Cubic Splines

1. Choose knots $\xi_1, ..., \xi_k$.
   ex: $\xi_1 = 20,\ \xi_2 = 25,\ \xi_3 = 30$, etc.
2. Run linear regression with
   - Intercept
   - Predictors $X,\ X^2,\ X^3$

- Predictors $h(X, \xi_i) = (X - \xi)_+^3 = \begin{cases} (X-\xi)^3 & X > \xi \\ 0 & \text{otherwise} \end{cases}$ for $i = 1, ..., k$

3. Resulting function is

$$y(X) = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \sum_{i=1}^{k} \beta_{k+3}(X - \xi)_+^3$$

$$y'(X) = \beta_1 + 2\beta_2 X + 3\beta_3 X^2 + \sum_{i=1}^{k} 3\beta_{k+3}(X - \xi)_+^2$$

$$y''(X) = 2\beta_2 + 6\beta_3 X + \sum_{i=1}^{k} 6\beta_{k+3}(X - \xi)_+$$

$$y'''(X) = 6\beta_3 + \sum_{i=1}^{k} 6\beta_{k+3}\mathbb{I}\{X > \xi\}$$

Terminology. These are sometimes called "regression splines" because they are fit with multiple linear regression.

- There is also "natural splines" which is "regression splines" plus a restriction that the prediction is linear (not cubic) for $X < \xi_1,\ X > \xi_k$. This one has a special basis with an intercept and $k + 3$ predictors.

- Last we study "smoothing splines" which find a function g that minimizes

$$\sum_{i=1}^{n}(y_i - g(X_i))^2 + \underbrace{\lambda \int_{-\infty}^{\infty} g''(t)^2 dt}_{\text{tuning/regularization parameter}}$$

  for a data set $(X_i,\ Y_i)_{1 \le i \le n}$

Smoothing splines. $\lambda = 0 \Rightarrow$ penalty has no effect so we fit the data perfectly.
$\lambda \to \infty \Rightarrow g''(t) = 0$ everywhere $\Rightarrow g(t) = at + b \Rightarrow g$ is the simple linear regression predictor.
$\lambda \in (0, \infty) \Rightarrow g$ is a natural cubic spline with knots at $X_1, ..., X_n$

In practice, λ controls the level of smoothness vs. wiggliness $\Rightarrow$ tune λ with a validation set or cross-validation.

Generalized additive model. We replace multiple linear regression

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_p X_{ip} + \epsilon_i$$

with

$$y_i = \beta_0 + \underbrace{f_1(X_{i1})}_{\text{nonlinear function}} + f_2(X_{i2}) + ... + f_p(X_{ip}) + \epsilon_i$$

There's a cool approach called "backfitting" where we learn each function f_i by predicting the "partial residual."

$\boxed{ex}$ $p = 3 \quad \hat{y}_i = \beta_0 + f_1(X_{i1}) + f_2(X_{i2}) + f_3(X_{i3})$
Set $\beta_0 = \bar{y}$.. Set $f_1,\ f_2,\ f_3 = 0$
Repeat until convergence:

1. Train f_1 to predict $y_i - \beta_0 - f_2(X_{i2}) - f_3(X_{i3})$
2. Train f_2 to predict $y_i - \beta_0 - f_1(X_{i1}) - f_3(X_{i3})$
3. Train f_3 to predict $y_i - \beta_0 - f_1(X_{i1}) - f_2(X_{i2})$

5.3 Regression Trees

$\boxed{ex}$ Hitters data. Predict salary by

1. Log transform data
2. Build tree

Recursive binary splitting.
Split each cell by finding the predictor X_j and cutpoint s that minimizes

$$\sum_{i:X_{ij}<s} (Y_i - \hat{Y}_{R_1})^2 + \sum_{i:X_{ij}\geq s} (Y_i - \hat{Y}_{R_2})^2$$

where $\hat{Y}_{R_1}$ is mean response for predictors $X_{ij} < S$ and $\hat{Y}_{R_2}$ is mean response for predictors $X_{ij} \geq S$

Rectangles

$$R_1 = \{X = \begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix} \mid x_j < s\}$$

Weakest link pruning.
Given a big tree T_0, find a subtree (T_0 with some branches removed) $T \subseteq T_0$ that minimizes

$$\sum_{m=1}^{|T|} \sum_{i:X_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

where R_m is a rectangle corresponding to a leaf, α is the penalty term, and $|T|$ is the number of leaves.
As we increase α from 0 to ∞ branches get pruned in a nested manner.

Classification tree.
Compare

$$f(x) = \beta_0 + \sum_{j=1}^{p} \beta_j X_j$$

vs.

$$f(x) = \sum_{m=1}^{M} c_m \mathbb{I}\{X \in R_m\}$$

Bagging.
Create B bootstrap data sets. Train decision trees. Average predictions.

$$\hat{f}_{\text{bag}}(X) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{(b)}(X)$$

Random Forest.
Create B bootstrap data sets. For each data set, uniformly randomly choose $\approx \sqrt{p}$ predictors and train a decision tree using only chosen predictors +

bootstrap data. Average predictions as before

$$\hat{f}(X) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{(b)}(X)$$

Works a little better than bagging.

5.4 Maximum Margin Classifiers

Question: what is a hyperplane?

In 2D, it is a line, a set of vectors $X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ such that $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$. Technically, this is an "affine" hyperplane since it doesn't necessarily pass through (0,0).

In p dimensions, a hyperplane is a set of vectors $X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix}$ such that $\beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_p X_p$.

Question: how far away is the origin from the hyperplane?

Divide through by $(\sum_{j=1}^{p} \beta_j^2)^{\frac{1}{2}} = \left|\left| \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} \right|\right|$

$\boxed{ex}$ $1 + 2X_1 + 3X_2 = 0$

Divide through by $\sqrt{13}$, we get the following hyperplane equation

$$\frac{1}{\sqrt{13}} + \frac{2}{\sqrt{13}}X_1 + \frac{3}{\sqrt{13}}X_2 = 0$$

For any point $X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, $\frac{1}{\sqrt{13}} + \frac{2}{\sqrt{13}}X_1 + \frac{3}{\sqrt{13}}X_2$ is the "signed distance" to the hyperplane.

Question: what is the distance from $X = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$ to the hyperplane?

Signed distance $= \frac{-4}{\sqrt{13}}$, which means distance $= \frac{4}{\sqrt{13}}$

- If $\beta_0 + \beta_1 X_1 + ... + \beta_p X_p > 0$ you lie on one side of the hyperplane.

- If $\beta_0 + \beta_1 X_1 + ... + \beta_p X_p < 0$ you lie on the opposite side of the hyperplane.
- If it works perfectly to separate data, this is called a "separating hyperplane."

We are given observations $(X_i, y_i)_{1 \leq i \leq n}$ where $y_i \in \{-1, +1\}$ (any binary outcome). We want a separating hyperplane.

$$\begin{cases} \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_p X_{ip} > 0 & \text{if } y_i = +1 \\ \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_p X_{ip} < 0 & \text{if } y_i = -1 \end{cases}$$

Equivalently,

$$y_i(\beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_p X_{ip}) > 0$$
$$\text{for each } i = 1, 2, ..., n$$

This could be used for out-of-sample predictions

$$\hat{y}_i = \text{sign}(\beta_0 + \beta_1 X_{i1} + ... + \beta_p X_{ip})$$
$$\text{for } i = n+1, n+2, ...$$

Let's write the optimization problem "maximum margin" solves.
Maximize the Euclidean distance from the nearest data points to the hyperplane m such that

$$\begin{cases} \sum_{j=1}^{p} \beta_j^2 = 1 \\ y_i(\beta_0 + \beta_1 X_{i1} + ... + \beta_p X_{ip}) \geq m \\ \qquad\qquad \text{for each } i = 1, ..., n \end{cases}$$

For data in "general position" (ex. with tiny jitter added to data points), there are exactly $p+1$ support vectors that lie a distance m from the hyperplane with at least one on both sides of the hyperplane.

1. Program the optimization and solve it on a computer (no closed form solution).
2. This results in the mathematical properties above.

5.5 Support Vector Classifiers

1. Solve the related optimization problem on your computer.

2. Maximize m such that

$$\begin{cases} \sum_{j=1}^{p} \beta_j^2 = 1 \\ y_i(\beta_0 + \beta_1 X_{i1} + ... + \beta_p X_{ip}) \geq m(1 - \epsilon_i) \quad \text{for } i = 1, ..., n \\ \epsilon_i \geq 0, \quad \sum_{i=1}^{n} \epsilon_i \leq C \leftarrow \text{tuning parameter} \end{cases}$$

- $\epsilon_i > 0$: Observation i violates the margin.
- $\epsilon i > 1$: Observation i lies on the wrong side of the hyperplane.

5.5.1 Support Vector Machines

This method is a flexible non-linear classifier.
Question: How do we do non-linear machine learning?
Idea #1: Add quadratics, cubics, etc.

$$f(x) = \beta_0 + \sum_{j=1}^{p} \beta_{j1} X_j + \sum_{j=1}^{p} \beta_{j2} X_j^2 \quad X = \begin{bmatrix} X_1 \\ \vdots \\ X_p \end{bmatrix}$$

$$\begin{cases} \text{if } f(x) > 0, \text{ guess } \hat{y} = +1 \\ \text{if } f(x) < 0, \text{ guess } \hat{y} = -1 \end{cases}$$

Idea #2: Use the "kernel trick," where we replace every inner product $< x, y >= \sum_{j=1}^{p} x_j y_j$ with a different nonlinear inner product

$$K(x, y) = \exp(-\gamma \sum_{j=1}^{p} (x_j - y_j)^2)$$

$$\text{for } \gamma > 0$$

$\boxed{ex}$ $K(x,y) = (1 + \sum_{j=1}^{p} x_j y_j)^d$
Then we use the prediction function

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i \exp(-\gamma \sum_{j=1}^{p} (x_j - x_{ij})^2)$$

or

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i (1 + \sum_{j=1}^{p} x_j, x_{ij})^d$$

or

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i \exp(-\gamma \sum_{j=1}^{p} |x_j - x_{ij}|)$$

5.5.2 Kernel Functions

There are 3 main kernel functions:

1. Inner product kernel

$$k(x,y) = < x, y > = \sum_{j=1}^{p} x_j y_j$$

2. Polynomial kernel

$$k(x,y) = (1 + < x, y >)^d = (1 + \sum_{j=1}^{p} x_j y_j)^d$$

here $d \in \{2, 3, ...\}$

3. Radial kernel

$$k(x,y) = \exp(-\gamma ||x - y||^2) = \exp(-\gamma \sum_{j=1}^{p} (x_j - y_j)^2)$$

How does the computer fit/optimize parameters $\beta_0, \alpha_1, ..., \alpha_n$? We choose parameters to optimize

$$\underbrace{L}_{\text{loss function}} = \underbrace{\sum_{i=1}^{n} \max\{0, 1 - y_i f(x_i)\}}_{\text{data misfit/hinge loss}} + \underbrace{\lambda ||f||_k^2}_{\text{penalty term } \lambda > 0}$$

$$||f||_k^2 = \sum_{i,l=1}^{n} \alpha_i \alpha_l K(x_i, x_l)$$

Where j is the index for different predictors, i, l to index observations. If $f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i k(x_1, x_i)$
Remarks:

- As $\lambda \nearrow \infty$, the optimization wants to make $||f||_k^2$ small $\Rightarrow$ all the α_i's are small $\Rightarrow$ collapses to a constant predictor $f(x) = \beta_0$.
- As $\lambda \searrow 0$, the optimization doesn't care about the size/complexity of α_i's $\Rightarrow$ perfect classification on training data.

So what is the margin? What are the support vectors?
The margin is the region $\{X \mid -1 \leq f(x) \leq 1\}$

(a) "X_i is on the margin" $[y_i f(x_i) = +1]$

$$\Leftrightarrow \begin{cases} y_i = +1 & f(x_i) = +1 \\ y_i = -1 & f(x_i) = -1 \end{cases} \qquad \boxed{\text{marginal}}$$

(b) "X_i is on the right side of the margin" $[y_i f(x_i) > +1]$

$$\Leftrightarrow \begin{cases} y_i = +1 & f(x_i) > +1 \\ y_i = -1 & f(x_i) < -1 \end{cases} \qquad \boxed{\text{good}}$$

(c) "X_i is on the wrong side of the margin" $[y_i f(x_i) < +1]$

$$\Leftrightarrow \begin{cases} y_i = +1 & f(x_i) < +1 \\ y_i = -1 & f(x_i) > -1 \end{cases} \qquad \boxed{\text{bad}}$$

Mathematical support means nonzero coefficients. Support vectors are all the data input points X_i with indices in

$$S = \{i \mid y_i x_i \leq +1\}$$
$$i = 1, 2, \ldots, n$$

In other words, anyone who is in or on the margins, or is misclassified is a support vector.

SVMs are "sparse"

$$\begin{aligned} f(x) &= \beta_0 + \sum_{i=1}^{n} \alpha_i k(x, x_i) \\ &= \beta_0 + \sum_{i \in S} \alpha_i k(x, x_i) \end{aligned}$$

This means $\alpha_i = 0$ if i is outside the support set/ X_i is on the right side of the boundary.

5.5.3 Gradient Descent

This is an algorithm for finding a (local) minimum of a function.
Idea: Use the gradient, which is the direction of steepest increase.
Algorithm:

1. Start at x_0
2. New guess:

$$x_{n+1} = x_n - \rho \nabla f(x_0) \quad \rho \in \mathbb{R},\ \rho \text{ is small}$$

Fact: If ρ is small enough, gradient descent is guaranteed to converge to a local minimum of f

6 Neural Networks

6.1 Introduction to Neural Networks

Question: What is a neural network?

$$\text{Input layer } x_i \rightarrow \text{ Hidden layer } A_j \rightarrow \text{ Output layer } f(x) \rightarrow y$$

- Arrows indicate function dependencies
  $f(x)$ is a function of A_1, A_2, A_3, A_4, A_5
  A_1 is a function of x_1, x_2, x_3, x_4

- Usually, these are very specific functional dependencies

$$f(x) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

- Output layer is a linear function of hidden neurons (in the layer preceding it)

For each $k = 1, 2, \ldots, K$,

$$A_k = g \cdot \left(\underbrace{w_{k0} + \sum_{j=1}^{p} w_{kj} x_j}_{\text{linear function}} \right)$$

Where g is a nonlinear activation function, or a rectified linear unit.

$$g(z) = \text{ReLu}(z) = \begin{cases} z & z > 0 \\ 0 & z \leq 0 \end{cases}$$

- The output layer does not have a nonlinear activation.
- The hidden layers do have a nonlinear activation.

Nowadays, we can code a very complicated prediction functions. The computer will do most of the work to train/optimize parameters based on a loss function that you specify.

Question 1: What parameters do we need to optimize?

$$f(x) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

Add all the arrows (weights) and the neurons they point to (biases).

- We have $K + 1$ parameters between the hidden and the output layers. Where K is the number of arrows.
- We have 25 parameters between the input and hidden layers in our class example. This adds up to a total of 31 parameters.

Question 2: What loss function could/should we use?

$$\min \sum_{i=1}^{n} (y_i - f(x_i))^2$$

which is the RSS given $(x_i, y_i)_{1 \leq i \leq n}$

"Labels"/responses are encoded as vectors. In the case of the MNIST dataset.

$$y = \begin{bmatrix} y_0 \\ \vdots \\ y_9 \end{bmatrix}$$

With a 1 indicating the correct digit, and 0s everywhere else. This is also known as One-hot encoding.
In the MNIST dataset, there are 60,000 training samples, and 10,000 testing images. ISLP trained using two hidden layers L_1 with 256 neurons and L_2 with 128 neurons.

Input layer	Hidden layer L_1	Hidden layer L_2	Output layer
x_0	$A_1^{(1)}$	$A_1^{(2)}$	$f_0(x) \to y_0$
x_1	$A_2^{(1)}$	$A_2^{(2)}$	$f_1(x) \to y_1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_p	$A_{k_1}^{(1)}$	$A_{k_2}^{(2)}$	$f_9(x) \to y_9$

$k_1 = 256,\ k_2 = 128,\ p = 784 = 28^2$
How many parameters?

$$(pk_1 + k_1) + (k_1 k_2 + k_2) + (k_2 \cdot 10 + 10) = 235,146$$

ReLu net

$$A_k^{(1)} = g\left(w_{k0}^{(1)} + \sum_{j=1}^{p} w_{kj}^{(1)} x_j\right)$$

$$A_l^{(2)} = g\left(w_{l0}^{(2)} + \sum_{k=1}^{K_1} w_{lk}^{(2)} A_k^{(1)}\right)$$

$$f_m(x) = \frac{e^{z_m}}{\sum_{l=0}^{9} e^{z_l}}$$

$$z_m = \beta_{m0} + \sum_{l=1}^{k_2} \beta_{ml} A_l^{(2)}$$

6.2 Convolutional Neural Nets

Overwhelmingly, convolutional neural nets are for 2-dimensional data.

Each input pixel will be denoted as X_{ivh} where

$$X_{i,v,h} = \begin{cases} i \in \{1,2,3\} & \text{input index (RGB)} \\ v \in \{1,2,\ldots,32\} & \text{vertical position} \\ h \in \{1,2,\ldots,32\} & \text{horizontal position} \end{cases}$$

Each output variable will be denoted as A_{ovh} where

$$A_{o,v,h} = \begin{cases} o \in \{1,2,3,4,5,6\} & \text{output index (RGB)} \\ v \in \{1,2,\ldots,32\} & \text{vertical position} \\ h \in \{1,2,\ldots,32\} & \text{horizontal position} \end{cases}$$

Convolutional filters

$$\text{Original Image} \rightarrow \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix}_{4\times 3}$$

Your eyes naturally sees little patches, say $\begin{bmatrix} g & h \\ j & k \end{bmatrix}$

$$\text{Convolutional Filter} \rightarrow \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}_{2\times 2}$$

These filters map from each input channel $i = 1, 2, 3$ to each output channel $o = 1, 2, 3, 4, 5, 6$. By standard, these are usually 2×2.

$$32 \times 32 \ (\times 3) \to 32 \times 32 \ (\times 6)$$
$$X_{i,v,h} \to A_{o,v,h}$$

The number of free parameters is (# of input channels)(# of output channels) (# of parameters in filter). In class example of $32 \times 32(\times 3) \to 32 \times 32(\times 6)$, we have $3(6)(4) = 72$ weight parameters.

$$\underline{\text{Convolved Image}}$$
$$\begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}_{3\times 2}$$

ex CNN architecture

Given a box denoting an image, the north and west side will have the number of pixels of that side. The south side will have $(\times 3)$ which denotes 3 channels, indicating intensity in RGB. The pattern goes convolve, pool, convolve, pool, ..., and eventually flatten.

Before applying the 2×2 convolutional filter to 32×32 image, add a horizontal and vertical level of zeros, temporarily yielding a 33×33 image before convolving. This is called zero padding.

Apply a 2×2 convolution to each channel separately, add them together, add a bias term b_1 and apply a ReLU activation.

$$A_{o,v,h} = \text{ReLU}\left(w_0 + \sum_{i=0}^{\text{3channels}} \begin{bmatrix} X_{i,v,h} w_{i11} & X_{i,v,h+1} w_{i12} \\ X_{i,v+1,h} w_{i21} & X_{i,v+1,h+1} w_{i22} \end{bmatrix} \right)$$

There are two types of pooling, max pooling and average pooling.

$$\underline{\text{Max pooling}}$$
$$\left[\begin{array}{cc|cc} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ \hline 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{array}\right]_{4\times 4} \to \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}_{2\times 2}$$

Average pooling

$$\left[\begin{array}{cc|cc} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ \hline 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{array}\right]_{4\times 4} \rightarrow \begin{bmatrix} 1.5 & 2.75 \\ 1.25 & 2.25 \end{bmatrix}_{2\times 2}$$

We can apply a softmax to make the final vector in the CNN pathway into probabilities.

$$\begin{bmatrix} z_1 \\ \vdots \\ z_k \end{bmatrix} \rightarrow \frac{e^{z_i}}{\sum_{k=1}^{K} e^{z_k}}$$

With 6 output channels ($\times 6$), each channel uses 2×2 convolutional filters on the 3 input channels and adds a bias parameter.

$$6[(3)(4) + 1] = 6 \cdot 13 = 78$$

With 12 output channels ($\times 12$), each channel uses 2×2 convolutional filters on the 6 input channels and adds a bias parameter.

$$12[(6)(4) + 1] = 300$$

On the flatten layers, we multiply the number of inputs to outputs and add bias parameters for the number of outputs we have.

7 Unsupervised Learning

Supervised learning means prediction, where we are given predictors

$$X_1, X_2, \ldots, X_p \rightarrow \text{ predict } y$$

Unsupervised learning means compression, where we find a sample data format that compresses observations $X_1, X_2, \ldots, X_n \in \mathbb{R}^p$ as accurately as possible.

Examples

- Principal component analysis
  Every data point requires an x and a y coordinate, giving $2n$ numbers to store, PCA creates a best-fit line and stores the points' location when projected on the line, allowing us to only store $n + 1$ numbers, the locations and the slope of the line.

- Clustering
  Create m clusters of data points and store only the cluster centers, we only have to store $2m$ numbers along with n booleans for which cluster the data point belongs to.

7.1 Principal Component Analysis

In PCA, predictors are called features.

1. Standardize the data by subtracting the mean and dividing by the standard deviation.

$$X_{ij} \leftarrow \tilde{X}_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j} \quad \text{where} \begin{cases} \mu_j &= \frac{1}{n}\sum_{i=1}^{n} X_{ij} \\ \sigma_j^2 &= \frac{1}{n-1}\sum_{i=1}^{n}(X_{ij} - \mu_j)^2 \end{cases}$$

2. Find first principal component, which is a linear combination that maximizes variance with normalized coefficients $\sum_{j=1}^{p} \phi_{ji}^2 = 1$.

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \cdots + \phi_{p1}X_p$$

   Largest variance means that the "loading" vector

$$\phi_1 = \begin{bmatrix} \phi_{11} \\ \phi_{21} \\ \vdots \\ \phi_{p1} \end{bmatrix}$$

   which solves the following maximization problem

$$\max_{\phi_{11},\ldots,\phi_{p1}} \underbrace{\frac{1}{n-1}\sum_{i=1}^{n}\left(\sum_{j=1}^{p}\phi_{j1}x_{ij}\right)^2}_{\text{variance of PC 1}} \text{ such that } \sum_{j=1}^{p}\phi_{j1}^2 = 1$$

   From this we can then define the scores for each data point

$$z_{i1} = \sum_{j=1}^{p}\phi_{j1}x_{ij} \quad \text{for } i = 1, 2, \ldots, n$$

3. Find second principal component that maximizes variance while being uncorrelated with Z_1.

$$Z_2 = \phi_{12}X_1 + \phi_{22}X_2 + \cdots + \phi_{p2}X_p$$

That is normalized, uncorrelated with Z_1, and maximizes variance.

$$\max_{\phi_{12},\ldots,\phi_{p2}} \frac{1}{n-1}\sum_{i=1}^{n}\left(\sum_{j=1}^{p}\phi_{j2}x_{ij}\right)^2$$

$$\text{such that } \sum_{j=1}^{p}\phi_{j2}^2 = 1 \text{ and } \text{Cov}(PC1, PC2) = 0$$

We can evaluate scores $z_{12}, z_{22}, \ldots, z_{n2}$ by the formula

$$z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \cdots + \phi_{p2}x_{ip}$$
$$\text{for } i = 1, 2, \ldots, n$$

ex Number of arrests per 100,000 residents for assault (X_1), murder (X_2), rape (X_3), and percent of population in urban areas (X_4). We have observations for each state ($n = 50$).

Two interpretations of PCA

1. Maximizing variance
2. Low-dimensional linear compression
   (a) PC1 is the line that lies closest to the data (in square Euclidean distance).
   (b) PC1 and PC2 form the plane that lies closest to the data.
   (c) Etc.

The linear compression that is generated by PCA is

$$x_{ij} \approx \sum_{m=1}^{M} z_{im}\phi_{jm}$$

This is called the "projection" onto first M principal components. We are storing all the numbers in the $n \times p$ data array in a more compressed format,

using the $n \times M$ score matrix.

An equivalent characterization of PCA is a minimization problem

$$\min \frac{1}{n} \sum_{j=1}^{p} \sum_{i=1}^{n} \left(x_{ij} - \sum_{m=1}^{M} z_{im} \phi_{jm} \right)^2$$

Here we are minimizing over the scores z_{im}, which can be arranged into a matrix $Z \in \mathbb{R}^{n \times M}$, also the loadings ϕ_{jm}, which can be organized into a matrix $\Phi \in \mathbb{R}^{p \times M}$. If you already know the scores, determining each of the PCs is an example of multiple linear regression.

Percent of variance explained. Also known as sum of squares decomposition.

$$\underbrace{\sum_{j=1}^{p} \sum_{i=1}^{n} x_{ij}^2}_{\text{sum of squares of data}} = \underbrace{\sum_{m=1}^{M} \sum_{i=1}^{n} z_{im}^2}_{\text{sum of squares of } M \text{ PCs}} + \underbrace{\sum_{j=1}^{p} \sum_{i=1}^{n} \left(x_{ij} - \sum_{m=1}^{M} z_{im} \phi_{jm} \right)^2}_{\text{residual sum of squares}}$$

Therefore we can calculate the R^2 statistic.

$$\begin{aligned} R^2 &= 1 - \frac{\text{RSS}}{TSS} \\ &= 1 - \frac{\sum_{j=1}^{p} \sum_{i=1}^{n} \left(x_{ij} - \sum_{m=1}^{M} z_{im} \phi_{jm} \right)^2}{\sum_{j=1}^{p} \sum_{i=1}^{n} x_{ij}^2} \\ &= \frac{\sum_{m=1}^{M} \sum_{i=1}^{n} z_{im}^2}{\sum_{j=1}^{p} \sum_{i=1}^{n} x_{ij}^2} \end{aligned}$$

This last term increases from 0 to 1 as we raise the number of principal components M. Often we choose M to explain $\geq 95\%$ of the variance.

7.2 Clustering

Motivation. We are given a data set $x_1, x_2, \ldots, x_n \in \mathbb{R}^p$. We want to divide the data into clusters which are specified by index sets $C_1, \ldots, C_K$ that satisfy

$C_1 \cup C_2 \cup \cdots \cup C_K = \{1, \ldots, n\}$	Every data point is in a cluster
$C_k \cap C_{k'} = \emptyset, \quad \text{for } k \neq k'$	Clusters are non-overlapping

In k-means clustering, we try to identify clusterings $C_1, \ldots, C_K$ that minimize the sum of square distances from the data points to the cluster centers

$$\min_{C_1,\ldots,C_K} \underbrace{\sum_{k=1}^{K} \sum_{i \in C_k} ||x_i - \bar{x}_k||^2}_{\text{sum of square distances}}, \quad \text{where} \quad \underbrace{\bar{x}_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i}_{\text{cluster center } k} \qquad (*)$$

Lloyd's algorithm. Applying k-means clustering.

1. Randomly assign a number from 1 to K to each of the observations. These serve as initial cluster assignments for the observations.

2. Iterate until the cluster assignments stop changing:

   (a) For each of the K clusters, compute the cluster center. The k'th cluster center is the vector of the p feature means for the observations in the k'th cluster.

   (b) Assign each observation to the cluster whose center is closest.

This algorithm systematically decreases the loss function $(*)$.

Dendrograms. A dendrogram is a hierarchical cluster where the height at which branches are fused measures the dissimilarity.

Hierarchical clustering. Hierarchical clustering forms a dendrogram from the bottom up. It starts with pairwise dissimilarities measured across the $\binom{n}{2}$ data pairs. Then at each step it fuses two clusters that are most similar or equivalently least dissimilar. There are several linkage rules, but the authors especially recommend the following.

- Complete linkage computes the maximal dissimilarity across all the members in two different clusters and uses this measure to determine which clusters to fuse next.

- Average linkage computes the average dissimilarity across all the members in two different clusters and uses this measure to determine which clusters to fuse next.

8 Review

Base definitions.

- RSS = residual sum of squares, for prediction model $f(X)$

$$\sum_{i=1}^{n}(y_i - f(X_i))^2$$

- MSE = mean square error

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - f(X_i))^2 = \frac{1}{n}\text{RSS}$$

- TSS = total sum of squares

$$\sum_{i=1}^{n}(y_i - \bar{y})^2$$

- R^2 = % of variance explained

$$1 - \frac{\text{RSS}}{\text{TSS}}$$

- r^2 = square correlation coefficient between X and y

$$\frac{\left[\sum_{i=1}^{n}(X_i - \bar{X})(y_i - \bar{y})\right]^2}{\sum_{i=1}^{n}(X_i - \bar{X})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Main topics.

1. Multiple linear regression
    - Simple linear regression
    - Ridge regression
    - Lasso
    - Polynomial regression
    - Ordered categorical variables

- Cubic splines

2. Logistic regression
3. Support vector machines
4. Gaussian mixture models
5. Tree-based classifiers
6. Neural nets
7. PCA
8. Clustering

Multiple linear regression.

1. Write down the model, count the variables

$$f(X) = \beta_0 + \sum_{j=1}^{p} \beta_j X_j$$

number of free parameters $= p + 1$
1 intercept β_0 p coefficients $\beta_1, \ldots, \beta_p$

2. Know the loss/objective function we are minimizing when we optimize the free parameters on a computer.

   (a) Choose $\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p$ to minimize RSS $\sum_{i=1}^{n}(y_i - f(X_i))^2$

   (b) Minimize RSS + $\lambda \sum_{j=1}^{p} \hat{\beta}_j^2$ (ridge regression)

   (c) Minimize RSS + $\lambda \sum_{j=1}^{p} |\hat{\beta}_j|$ (lasso)

   We often use $\lambda \in [0, \infty]$ to denote the non-negative "penalty parameter"

Question: When do we get coefficients exactly $= 0$ for a regularization parameter $\lambda < \infty$?

- Lasso
- Support vector machine

- NOT ridge regression

Simple linear regression
Multiple linear regression with 1 predictor

$$f(X) = \beta_0 + \beta_1 X$$

Minimizing RSS can be done explicitly

$$\begin{aligned}\hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{X} \\ \hat{\beta}_1 &= \hat{r}_{xy} \frac{\hat{\sigma}_y}{\hat{\sigma}_x} \\ &= \frac{\sum_{i=1}^n (X_i - \bar{X})(y_i - \bar{y})}{\sum_{i=1}^n (X_i - \bar{X})^2}\end{aligned}$$

Sometimes when X is a scalar, there's no general relationship between R^2 and r^2 BUT

$$\begin{cases} \text{for simple linear regression} & R^2 = r^2 \\ \text{for multiple linear regression} & R^2 \geq r_{yx}^2 \end{cases}$$

Where r^2 is square correlation coefficient between y and any X_j. Also note that more variables means better model.

Nonlinear prediction
When X is scalar, we often do nonlinear prediction $\rightarrow$ multiple linear regression with new nonlinear functions of X that we define ourselves.

1. Polynomial regression $f(X) = \beta_0 + \beta_1 X + \cdots + \beta_\alpha X^d$
   where d is the degree of the polynomial and there are $\alpha + 1$ free parameters

2. Ordered categorical variables

   $$\begin{aligned} f(X) =& \beta_1 \mathbb{I}\{X \leq \xi_1\} + \beta_2 \mathbb{I}\{\xi_1 < X \leq \xi_2\} + \cdots + \beta_k \mathbb{I}\{\xi_{k-1} < X \leq \xi_k\} \\ &+ \beta_{k+1} \mathbb{I}\{X > \xi_k\} \end{aligned}$$

3. Cubic splines

   $$f(X) = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \sum_{j=1}^K \beta_{j+3} (X - \xi_j)_+^3$$

   Here we have $k + 4$ free parameters.

Cubic splines

Choose $f(X)$ to minimize RSS $+ \lambda \int_{-\infty}^{\infty} |f''(t)|^2 dt$ (smoothing splines)

This results in a cubic spline model where

$$\begin{cases} \text{Knots } \xi_1 < \xi_2 < \cdots < \xi_n \text{ are ordered data pts } X_i \text{ for } i = 1, \ldots, n \\ f(X) \text{ is linear for } X \leq \xi_1 \\ f(X) \text{ is linear for } X \geq \xi_n \end{cases}$$

Question: when does cubic splines perfectly fit all the data points?

When $\lambda = 0$ and only the

$$\begin{cases} \text{Best test accuracy for intermediate } \lambda \in (0, \infty) \\ \text{As } \lambda \downarrow 0, f(X) \to \text{Cubic splines model that perfectly fits data } (X_i, y_i) \text{ for } i = 1, \ldots, n \\ \text{As } \lambda \uparrow \infty, f''(X) \to 0 \end{cases}$$